ESP32 CLOUD INTEGRATED PATIENT MONITORING SYSTEM

Submitted by

Dhanusri VEERAPPAN

ABSTRACT

This project involves developing an cloud integrated patient monitoring system capable of collecting, processing, and analyzing temperature, heart rate and SpO2 data in real time. The system integrates sensors like DHT11,MAX30100/30102 with microcontroller such as ESP32, leveraging Wi-Fi for connectivity.

The data is transmitted to cloud platforms for storage, visualization, and remote access. Advanced features include real-time alerts, automated and data analytics. Applications range from industrial temperature control to environmental monitoring, ensuring scalability, reliability, and energy efficiency through IoT integration.

The "Cloud-Integrated Patient Monitoring System" is a cutting-edge solution designed to monitor and manage specific human data in real-time. This system combines Internet of Things (IoT) sensors with cloud technology to provide a scalable, efficient, and reliable approach to temperature management across various environments, such as industrial facilities, medical storage, and smart homes. IoT-enabled sensors capture temperature readings and transmit them to a cloud platform via wireless networks.

The cloud infrastructure processes, stores, and visualizes data, enabling users to access temperature metrics through web or mobile applications. Advanced features, including real-time alerts and historical data analytics, allow users to take immediate corrective actions and optimize environmental conditions.

The system is designed to ensure high accuracy, energy efficiency, and robust security for sensitive data. By leveraging cloud computing and IoT, this project demonstrates an innovative way to enhance temperature monitoring, offering enhanced convenience, safety, and efficiency for diverse applications.

1. INTRODUCTION

Cloud-integrated patient monitoring systems represent a significant advancement in modern healthcare technology. These systems combine the capabilities of the Internet of Things (IoT) and cloud computing to enable real-time monitoring of patient health. By connecting sensors and microcontrollers to cloud platforms, patient data such as temperature, heart rate, and environmental conditions can be collected, processed, and accessed remotely by healthcare providers.

The demand for continuous and efficient health monitoring has grown due to the increasing prevalence of chronic illnesses and the need for proactive patient care. Traditional monitoring methods often require physical presence and are limited in scope, making them less effective in addressing the needs of today's healthcare systems. The study suggests a method whereby a patient's health status can be determined and health-promoting messages are delivered in a transparent manner through a wireless bodyarea network they are able to communicate with medical services. To understand their healthcare system potential, however, a multidisciplinary effort like cloud computing is necessary, and this will lead to in a new era of advanced healthcare services. The proposed approach makes the delivery of expert-based medical treatment through the cloud-based healthcare system more possible and practical. Data is collected and sent to a central cloud platform in a patient health monitoring system that incorporates cloud computing from a variety of health devices, including medical sensors and wearables. [6]

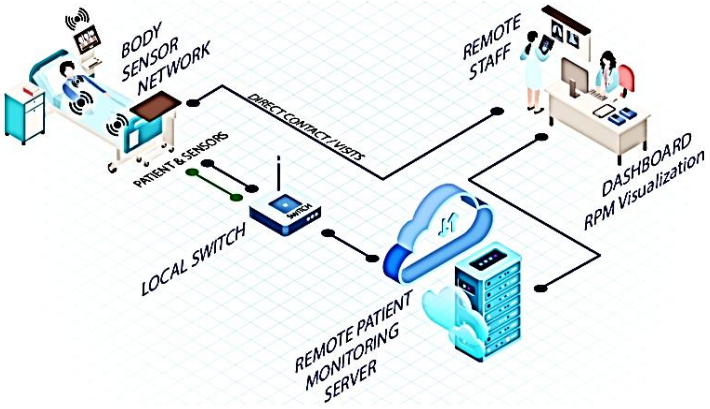

Figure 1 - Concept of the Cloud Integrated Monitoring System

1.1 PROBLEM FORMULATION

The healthcare industry faces a critical challenge in ensuring continuous and efficient monitoring of patients, particularly those with chronic illnesses or critical conditions. Traditional patient monitoring systems are often confined to hospital environments and rely on manual observation

and data recording. These systems are not only resource-intensive but also limited in their ability to provide real-time updates to caregivers and healthcare providers.

With the increasing prevalence of chronic diseases, an aging population, and the growing need for home-based healthcare solutions, there is a pressing demand for innovative systems that enable remote and real-time patient monitoring. Additionally, existing systems often fail to integrate multiple data points, resulting in fragmented health information and delayed responses to emergencies.

This problem is compounded in rural and underserved regions, where access to healthcare facilities is limited, and continuous monitoring is nearly impossible. A lack of timely intervention in critical situations can lead to severe health complications or fatalities.

To address these issues, a cloud-integrated patient monitoring system is required. Such a system would:

1. Facilitate the continuous collection of vital patient data using sensors.
2. Leverage cloud technology to store, analyze, and share data in real time.
3. Send automated alerts to caregivers and healthcare providers during emergencies.
4. Be cost-effective, scalable, and user-friendly for widespread adoption.

The formulation of this problem underscores the necessity for a solution that bridges the gap between traditional healthcare systems and modern technological capabilities, ensuring better health outcomes and improved quality of care.

1.2 OBJECTIVES

The primary objective of this project is to develop a system that can monitor patient vitals, such as temperature and pulse rate, in real-time. This ensures continuous tracking of health data without requiring the constant presence of a healthcare provider.

Another key goal is to integrate the system with cloud technology to store and manage patient data effectively. This allows healthcare professionals and caregivers to access the information remotely at any time, facilitating better decision-making and timely interventions.

The project also aims to send automated alerts during emergencies, ensuring immediate attention to critical situations. This feature helps reduce response times and improves patient outcomes.

The motivation behind developing an IoT-Based Remote Patient Monitoring System stems from the growing need to enhance patient care outside of traditional healthcare settings. With the global population steadily increasing and chronic health conditions becoming more prevalent, there is mounting pressure on healthcare systems to provide comprehensive care that extends beyond

hospital walls. Long-term care patients, individuals with chronic diseases such as diabetes and heart conditions, and those recovering from surgical procedures benefit greatly from continuous oversight that reduces the frequency of in-person check-ups and hospital visits. Implementing remote patient monitoring using IoT technology can bridge the gap between healthcare providers and patients by allowing medical professionals to receive timely data and alerting when necessary, thus preventing complications and improving outcomes[9].

1.3 BLOCK DIAGRAM

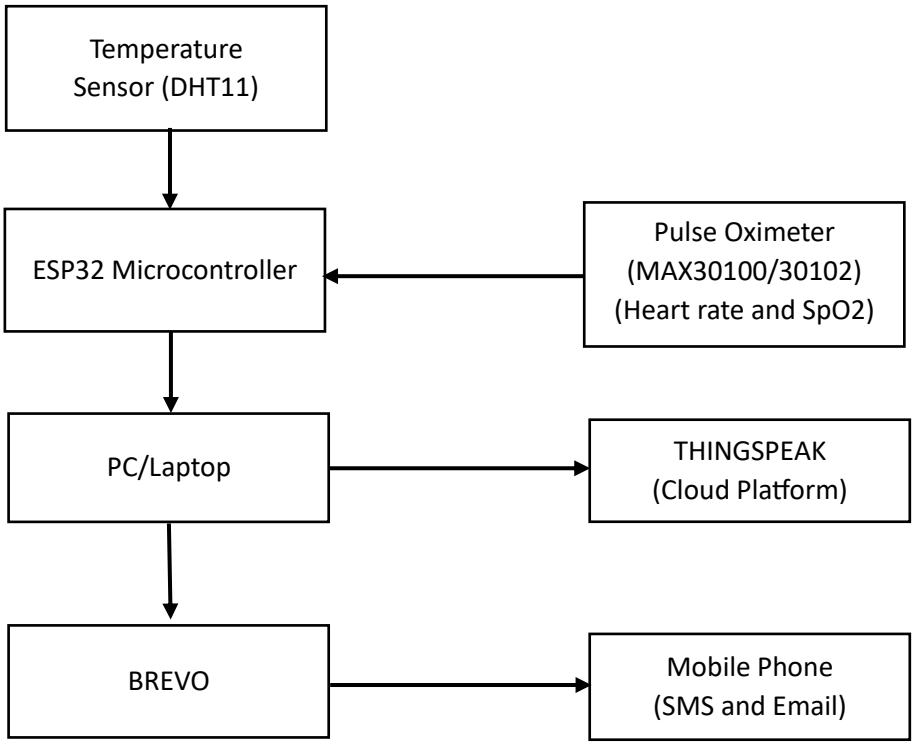

Figure 2 - Architecture

1.4 WORKING:

The block diagram illustrates the architecture of a cloud based patient monitoring system designed to measure vital parameters like temperature, heart rate, and oxygen saturation (SpO2). At the core of the system is the ESP32 microcontroller, which serves as the central processing unit. The temperature sensor (DHT11) and a pulse oximeter (MAX30100/MAX30102) are connected to the ESP32 to measure body temperature, heart rate, and SpO2 levels. These sensors provide real-time data, which the ESP32 processes and forwards for further analysis and monitoring.

The ESP32 sends the collected data to a PC or laptop for local processing and storage. From there, the data is uploaded to the cloud platform, ThingSpeak, enabling remote monitoring and visualization. This cloud integration allows healthcare professionals or caretakers to access the patient's vital data remotely in real time.

In case of any abnormal readings, the system sends alert messages via Brevo, which integrates with mobile devices to notify users through SMS or email. This ensures prompt alerts to caregivers or emergency contacts, facilitating timely intervention in critical situations.

1.5 LITERATURE SURVEY

RESEARCH PAPERS	TECHNIQUE USED	LIMITATIONS
D. Akhil, M. V. Vardhan, K. V. Reddy, C. Preetham and N. Sangeeta, "Iot Based Icu Patient Monitoring Smart System," *2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, Vellore, India, 2024, pp. 1-6, doi: 10.1109/AIIoT58432.2024.10574787	IoT-based ICU monitoring using multiple sensors integrated with cloud platforms.	Primarily focuses on ICU environments and lacks integration with alert messaging via SMS/Email (e.g., Brevo).
U. Gada, B. Joshi, S. Kadam, N. Jain, S. Kodeboyina and R. Menon, "IOT based Temperature Monitoring System," *2021 4th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)*, NaviMumbai, India, 2021, pp. 1-6, doi: 10.1109/ICNTE51185.2021.9487691	IoT-based temperature monitoring using sensors and cloud storage.	Limited to temperature monitoring only, no integration of SpO2 or heart rate measurements.
Jella Bharath Kumar, Banothu Charan , Guglavanth Vinod Kumar , Choudhary Sunil, Dr.P.Sumithabhashini," IoT Based Temperature Monitoring System UsingESP8266 & Blynk App", Volume 6, Issue 1, January-February 2024	Temperature monitoring using ESP8266 and the Blynk app for remote access.	Focuses only on temperature monitoring, lacks multi-parameter monitoring and advanced cloud visualization like ThingSpeak.
V. S, M. Jagadeeswari, R. R, M. Balasenduran, A. U. S and T. Akileshwaran, "IoT Based Automatic Temperature Detection and Visitor Recognition System," *2023 4th International Conference on Smart Electronics and Communication (ICOSEC)*, Trichy, India, 2023, pp. 392-397,doi: 10.1109/ICOSEC58147.2023.10276022.	IoT-based temperature detection with visitor recognition and automation.	Visitor recognition is irrelevant to patient monitoring, lacks pulse oximeter integration and real-time alerts.

Budijono, Santoso & Felita,"Smart Temperature Monitoring System Using ESP32 and DS18B20",2021,IOP Conference Series: Earth and Environmental Science.,794. 012125. 10.1088/1755-1315/794/1/012125.	Smart temperature monitoring using ESP32 and DS18B20.	Limited to temperature monitoring; no alert messaging or integration with patient-centric platforms.
Jeyavathana, Dr. Beulah and Ajay, Sakili and Srinivas, Doma Pavan, Integration of Cloud-Based Patient Healthcare Monitoring System (November 22, 2024)	Cloud-based healthcare monitoring system for patient data integration.	General healthcare focus; lacks specific sensor data collection (e.g., SpO2 or heart rate) and real-time alerts via Brevo.
Sultana, Salma & Rahman, Sadia & Rahman, Md & Chakraborty, Narayan & Hasan, Tanveer, 2021, "An IoT Based Integrated Health Monitoring System" ,549-554.	Integrated health monitoring using IoT for multi-parameter data collection.	Lacks a detailed focus on real-time alerts and does not integrate SMS/Email alerts for critical conditions
C. Chakraborty and A. Kishor, "Real-Time Cloud-Based Patient-Centric Monitoring Using Computational Health Systems," in IEEE Transactions on Computational Social Systems, vol. 9, no. 6, pp. 1613-1623, Dec. 2022, doi: 10.1109/TCSS.2022.3170375.	Cloud-based patient monitoring using computational systems	Focuses more on computational systems and cloud analytics, lacks implementation of specific sensor systems like DHT11 or MAX30100
IOT-Based Remote Patient Monitoring System D Prasanth, P Karthikeyan, N Yughesh, V Vishva, Department of Computer Science and Engineering, Madagadipet, Puducherry, India, International Journal Of Innovative Research In Technology, December 2024 | IJIRT | Volume 11 Issue 7 | ISSN: 2349-6002 IJIRT 170786	Remote patient monitoring using IoT for healthcare data transmission.	Generalized approach to remote monitoring without specific focus on temperature or SpO2 integration and Brevo-based alert mechanisms

Table 1 – Literature survey

2. COMPONENTS

2.1 DHT11 TEMPERATURE SENSOR

DHT11 is a low-cost digital sensor for sensing temperature and humidity. This sensor can be easily interfaced with any micro-controller such as Arduino, Raspberry Pi etc… to measure humidity and temperature instantaneously.

DHT11 humidity and temperature sensor is available as a sensor and as a module. The difference between this sensor and module is the pull-up resistor and a power-on LED. DHT11 is a relative humidity sensor. To measure the surrounding air this sensor uses a Thermistor and a capacitive humidity sensor.[10]

Features and Specifications

1. Temperature Measurement:
    - Measurement range: 0°C to 50°C
    - Accuracy: ±2°C
2. Operating Voltage: 3.3V to 5.5V
3. Sampling Rate: 1 reading per second (1 Hz)

Working Principle

Temperature Measurement:
The thermistor measures temperature by detecting changes in resistance due to varying heat levels.

The sensor processes the raw data internally and outputs it as a digital signal, which can be read directly by microcontrollers like the ESP32.

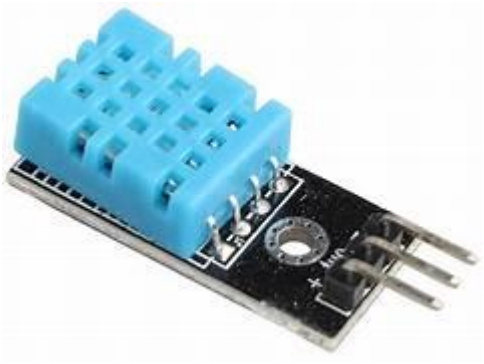

Figure-3 DHT11 Sensor

Pin Layout (Standard DHT11 Module)

For modules that include a built-in pull-up resistor and are mounted on a small PCB:

1. VCC (Leftmost pin).
2. Data (Middle pin).

3. GND (Rightmost pin).

2.2 MAX30100/30102 PULSE OXIMETER

The **MAX30100** and **MAX30102** are integrated pulse oximeter and heart rate sensor modules. These sensors are widely used in wearable health devices, fitness trackers, and medical monitoring systems. The MAX30100 is an integrated pulse oximetry and heartrate monitor sensor solution. It combines two LEDs, a photodetector, optimized optics, and low-noise analog signal processing to detect pulse oximetry and heart-rate signals[11]

Features and Specifications

1. **SpO2 Measurement**:
    - Measures oxygen saturation in the blood.
    - Uses infrared (IR) and red light emitters to calculate SpO2.
2. **Heart Rate Monitoring**:
    - Detects pulse rate using a photoplethysmogram (PPG).
    - Accurate even during slight movement.
3. **Operating Voltage**:
    - Typically operates at 1.8V for internal logic and 3.3V for the I2C interface.
4. **I2C Communication Protocol**:
    - Simplifies integration with microcontrollers like ESP32.
5. **Low Power Consumption**:
    - Optimized for battery-powered applications.

Working Principle

The MAX30100/30102 uses photoplethysmography (PPG), a non-invasive optical technique. The sensor emits two wavelengths of light (red and IR) through the skin and detects the amount of light absorbed by blood vessels. The difference in absorption between oxygenated and deoxygenated blood is used to calculate SpO2, while pulse rate is derived from the periodic changes in light absorption caused by blood flow.

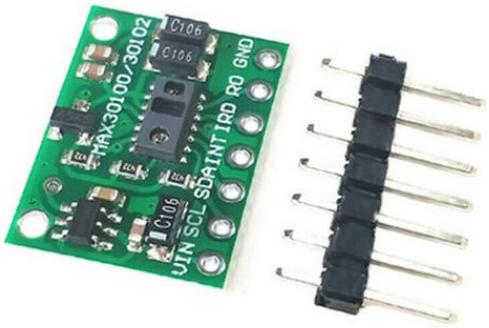

Figure-4 MAX30100/30102 Sensor

Pin Layout of MAX30100/30102

The **MAX30100/30102** sensor typically has 6 pins, with specific functionalities for power, communication, and grounding. Below is a detailed explanation of each pin:

1. **VIN (Power Input)**
    - Provides power to the sensor.Voltage range: 1.8V to 3.3V (typical operating voltage: 3.3V).
2. **GND (Ground)**
    - Connects the sensor to the system's ground.
3. **SCL (Serial Clock Line)**
    - Used for the I2C communication clock signal.
4. **SDA (Serial Data Line)**
    - Used for the I2C communication data signal.
5. **INT (Interrupt)**
    - Generates an interrupt signal to notify the microcontroller about data availability . Optional to use but can enhance efficiency in certain applications.
6. **IRD (Infrared Drive)**
    - Controls the infrared LED drive current .Typically manage d internally.

2.3 ESP32 MICROCONTROLLER

The ESP32 is a powerful and versatile microcontroller developed by Espressif Systems. It is widely used in IoT applications due to its robust performance, extensive features, and low power consumption. Equipped with integrated Wi-Fi and Bluetooth capabilities, the ESP32 is a popular choice for projects requiring connectivity, real-time processing, and remote control.

Its power efficiency, with options like deep sleep mode, makes it suitable for battery-powered devices. In healthcare projects, such as a Cloud Integrated Patient Monitoring System, the ESP32 acts as the central unit, collecting data from sensors, processing it, and transmitting it to the cloud via Wi-Fi. This enables real-time monitoring and remote access to vital patient data, significantly improving healthcare outcomes.

The ESP32’s combination of high performance, integrated connectivity, and cost-effectiveness makes it an essential component in modern IoT solutions, bridging the gap between devices and the digital world.

Key Features

1. Dual-Core Processor:
    - Powered by two Xtensa LX6 CPUs operating up to 240 MHz.
    - High processing power suitable for multitasking.
2. Connectivity:
    - Integrated Wi-Fi (802.11 b/g/n) for seamless internet connectivity.
    - Built-in Bluetooth 4.2 and BLE (Bluetooth Low Energy) for wireless communication.
3. GPIO (General Purpose Input/Output) Pins:
    - 36 GPIO pins for interfacing with sensors, actuators, and other peripherals.
4. Analog and Digital I/O:
    - Multiple ADC (Analog-to-Digital Converters) channels.Support for digital communication protocols like I2C, SPI, UART, and PWM.
5. Power Efficiency:
    - Features various power modes, including deep sleep mode, making it ideal for battery-powered applications.
6. Memory:
    - 520 KB SRAM and 4 MB Flash memory, allowing complex applications to run efficiently.

.

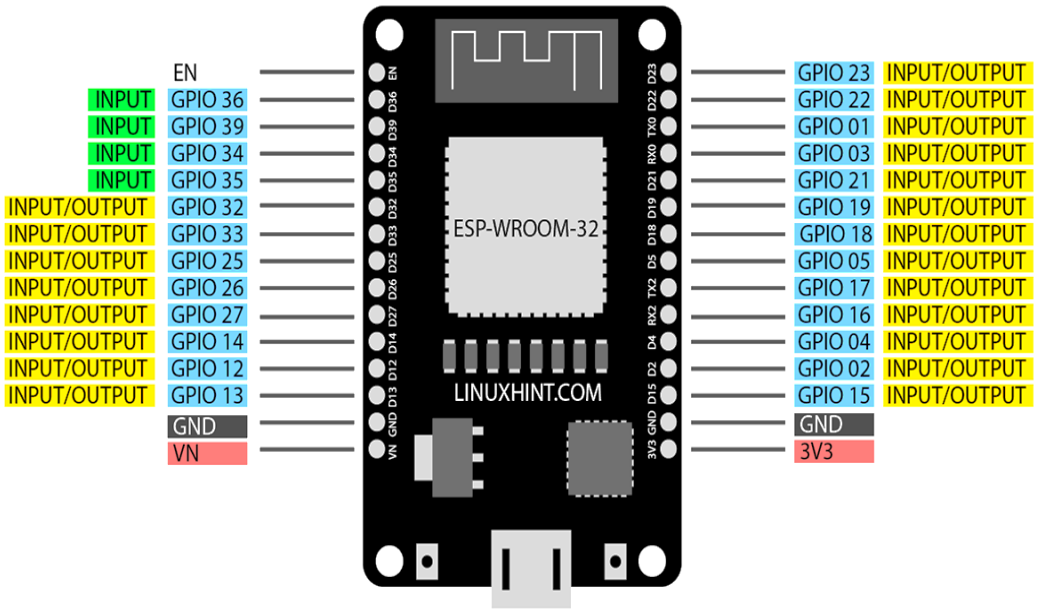

Figure-5 ESP32 Microcontroller

3. PROGRAMMING TOOL

3.1 ARDUINO IDE

The Arduino Integrated Development Environment (IDE) is a user-friendly, open-source software platform designed for writing, compiling, and uploading code to Arduino-compatible boards. It serves as the primary tool for programming microcontrollers and development boards like Arduino, ESP32, and ESP8266. The IDE simplifies the process of coding and debugging for beginners and professionals alike, making it a popular choice for IoT, robotics, and embedded systems projects.

The Arduino Integrated Development Environment (IDE) is a user-friendly software platform designed for writing, compiling, and uploading code to Arduino microcontrollers. It supports a variety of Arduino boards, making it easy to develop and test embedded systems projects. The IDE features a simple text editor, code auto-completion, and syntax highlighting, which help users write and debug programs efficiently.

It also includes built-in libraries and tools for accessing and controlling hardware components like sensors, motors, and displays. The Arduino Software (IDE) makes it easy to write code and upload it to the board offline. We recommend it for users with poor or no internet connection. This software can be used with any Arduino board.

There are currently two versions of the Arduino IDE, one is the IDE 1.x.x and the other is IDE 2.x. The IDE 2.x is new major release that is faster and even more powerful to the IDE 1.x.x. In addition to a more modern editor and a more responsive interface it includes advanced features to help users with their coding and debugging [12]

3.2 LIBRARIES USED

1.ThingSpeak Library

The ThingSpeak library is used for sending data to the ThingSpeak IoT platform. ThingSpeak allows real-time data collection, visualization, and analysis for IoT applications. It is widely used for logging and monitoring sensor data online.

Key Features:

- Easy integration with ESP32 and Arduino-based projects.
- Functions to write sensor data to channels and read data from fields.
- Secure data transmission using HTTPS.

Installation:

- Available in the Arduino IDE Library Manager.

Example Usage:

```
#include <ThingSpeak.h>
// Initialize ThingSpeak with an available client.
ThingSpeak.begin(client);
ThingSpeak.writeFields(channelID, writeAPIKey);
```

Figure-6 Implementation of ThingSpeak library

2. DHT Sensor Library

The DHT Sensor Library is used for interfacing with DHT11 and DHT22 temperature and humidity sensors. It simplifies communication with these sensors and provides ready-to-use functions for data retrieval.

Key Features:

- Supports DHT11 and DHT22 sensors.
- Functions for reading temperature and humidity.
- Handles timing issues inherent to DHT sensors.

Installation:

- Available in the Arduino IDE Library Manager.

Example Usage:

```
#include <DHT.h>
DHT dht(DHTPIN, DHTTYPE);
dht.begin();
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
```

Figure-7 Implementation of DHT Sensor library

3. MAX30100 Library

The MAX30100 library is used for interfacing with the MAX30100 or MAX30102 pulse oximeter and heart rate sensor. It provides functions for reading heart rate (bpm) and blood oxygen levels (SpO2).

Key Features:

- Real-time data for heart rate and SpO2.
- Functions for initializing and updating the sensor.
- Handles calibration and error conditions.

Installation:

- Download the library from GitHub or the Arduino IDE Library Manager.

Example Usage:

```
#include <MAX30100.h>
MAX30100 sensor;
sensor.begin();
sensor.update();
float heartRate = sensor.getHeartRate();
float SpO2 = sensor.getSpO2();
```

Figure-8 Implementation of MAX30100 library

4. WiFi Library

The WiFi library is essential for enabling ESP32 to connect to a Wi-Fi network. It provides functions for connecting, reconnecting, and managing Wi-Fi communication.

Key Features:

- Handles both station and access point modes.
- Functions for network scanning and connection management.
- Provides IP address, signal strength, and connection status.

Installation:

- Pre-installed with the ESP32 board package in the Arduino IDE.

Example Usage:

```
#include <WiFi.h>
WiFi.begin("SSID", "PASSWORD");
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Connecting...");
}
```

Figure-9 Implementation of WiFi library

5. Brevo (Sendinblue) Library

The Brevo (formerly Sendinblue) API enables sending emails or SMS notifications programmatically. While Brevo doesn't have an official Arduino library, HTTP requests are used for integration.

Key Features:

- Send transactional emails or SMS directly from the ESP32.
- Customizable notifications with subject and message content.
- Secure API key-based authentication.

Usage:

You create HTTP requests using the ESP32 WiFi or HTTPClient libraries.

Example Usage:

```
#include <HTTPClient.h>
HTTPClient http;
http.begin("https://api.brevo.com/v3/smtp/email");
http.addHeader("Content-Type", "application/json");
http.addHeader("api-key", "YOUR_API_KEY");
String json = "{\"sender\": {\"email\": \"sender@example.com\"},
              \"to\": [{\"email\": \"recipient@example.com\"}],
              \"subject\": \"Alert\",
              \"htmlContent\": \"<h1>Warning</h1>\"}";
http.POST(json);
```

Figure-10 Implementation of Brevo (Sendinblue) library

4. CONNECTION DETAILS

4.1 PIN DETAILS

1. ESP32 Pinouts:

- ESP32 has multiple GPIO pins. The ones used for sensors (DHT11, MAX30100) and other peripherals (like OLED or TFT display) are essential.

2. Sensor Connections:

a. DHT11 (Temperature and Humidity Sensor):

- VCC → 3.3V or 5V (Check your sensor's operating voltage).
- GND → GND.
- DATA → Connect to a digital pin, e.g., GPIO4. Use a 10kΩ pull-up resistor between the DATA and VCC lines.

b. MAX30100/30102 (Pulse Oximeter):

- VIN → 3.3V.
- GND → GND.
- SCL → GPIO22 (Default I2C clock pin on ESP32).
- SDA → GPIO21 (Default I2C data pin on ESP32).

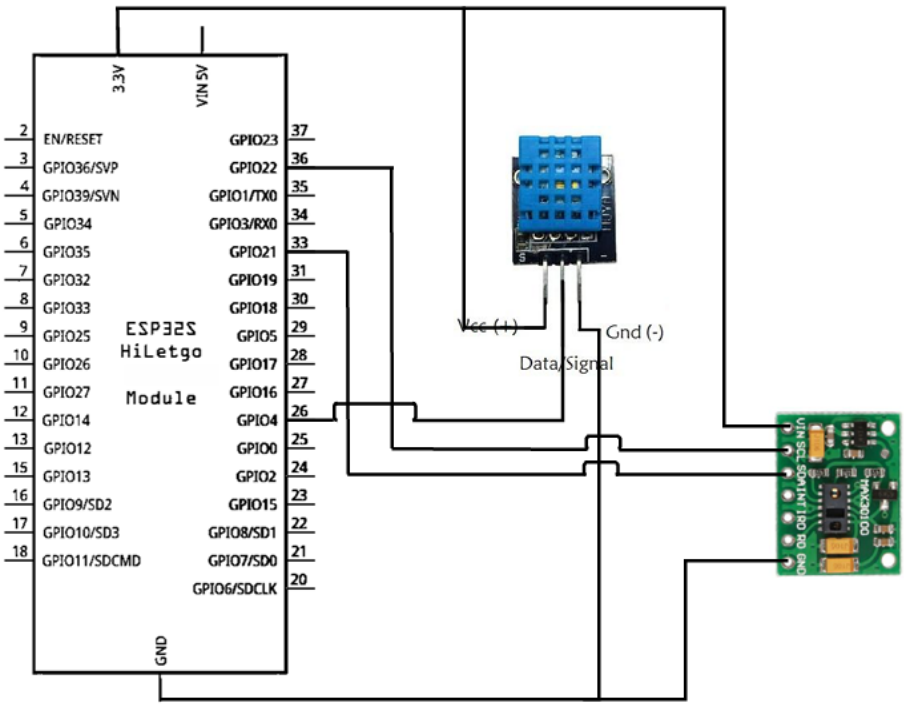

Figure-11 Pin connection

Connection Details for Cloud Integrated Patient Monitoring System

The Cloud Integrated Patient Monitoring System connects multiple sensors to an ESP32 microcontroller to monitor vital signs such as temperature, humidity, heart rate, and blood oxygen levels. The DHT11 temperature and humidity sensor is connected to the ESP32 with its data pin linked to GPIO4, while its VCC and GND pins are connected to the 3.3V and GND pins on the ESP32, respectively.

The MAX30100 pulse oximeter sensor communicates with the ESP32 using the I2C protocol. Its SDA (data) and SCL (clock) pins are connected to the ESP32's GPIO21 and GPIO22, the default I2C pins. The sensor is powered by connecting its VIN pin to the 3.3V pin on the ESP32 and its GND to the ESP32's ground.

This configuration ensures efficient data collection and communication between the sensors and the ESP32. The ESP32 then transmits the data to cloud platforms like ThingSpeak using its Wi-Fi module for remote monitoring and analysis. The entire setup is powered via the ESP32's micro-USB port or through the VIN and GND pins using a 5V power supply. Components are wired using jumper cables and mounted on a breadboard for neat and modular assembly.

4.2 Source code:

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>
#include <Wire.h>
#include <MAX30100.h>
#include <HTTPClient.h>
```

```
#include <ArduinoJson.h>

// Define your Wi-Fi credentials and ThingSpeak settings
const char* ssid = "Sri";
const char* password = "dhanu002";
unsigned long channelID = 2786321;
const char* writeAPIKey = "7OBCFISEX2YAWYHA";

// DHT11 sensor setup
#define DHTPIN 4          // Pin connected to DHT11
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// MAX30100 setup
MAX30100 pox;  // Declare the MAX30100 object
float heartRate, SpO2;

// Brevo (Sendinblue) API settings for email/SMS
const String brevoAPI = "https://api.brevo.com/v3/smtp/email";
const String apiKey = "xkeysib-
4348bf58ba5fe424605e23890337c384800e90cf078fb30ea063a4dcee5d1ed3-J6tP9t8zFCbdth0y";

// Time intervals
unsigned long lastTime = 0;
unsigned long interval = 10000; // 10 seconds for data upload

// Thresholds for triggering alerts
float temperatureThreshold = 37.5;
float heartRateLowThreshold = 60;
float heartRateHighThreshold = 100;
float SpO2Threshold = 90;

void setup() {
Serial.begin(115200);
dht.begin();// Initialize DHT11
Wire.begin();// Initialize MAX30100
pox.begin();  // Start the MAX30100 sensor
```

```
// Connect to Wi-Fi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
delay(1000);
Serial.println("Connecting to WiFi...");
}
Serial.println("Connected to WiFi");

// Initialize ThingSpeak with WiFiClient
WiFiClient client;
ThingSpeak.begin(client);
}

void loop() {
// Read sensor data
float temperature = dht.readTemperature();
pox.update();  // Update the MAX30100 sensor data
heartRate = pox.getHeartRate();
SpO2 = pox.getSpO2();

// Check if the readings are valid
if (isnan(temperature) || isnan(heartRate) || isnan(SpO2)) {
Serial.println("Failed to read from sensors!");
return;
}

// Display sensor readings
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.print(" °C, Heart Rate: ");
Serial.print(heartRate);
Serial.print(" bpm, SpO2: ");
Serial.println(SpO2);

// Upload to ThingSpeak every interval
if (millis() - lastTime > interval) {
lastTime = millis();
```

```
ThingSpeak.setField(1, temperature);
ThingSpeak.setField(2, heartRate);
ThingSpeak.setField(3, SpO2);

int httpCode = ThingSpeak.writeFields(channelID, writeAPIKey);
if (httpCode == 200) {
Serial.println("Data uploaded to ThingSpeak");
} else {
Serial.println("Failed to upload to ThingSpeak");
}
}
// Trigger Brevo notification if SpO2, heart rate, or temperature is abnormal
if (SpO2 < SpO2Threshold || heartRate < heartRateLowThreshold || heartRate >
heartRateHighThreshold || temperature > temperatureThreshold) {
sendBrevoAlert(temperature, heartRate, SpO2);
}

delay(2000); // Delay for a while before the next reading
}
// Function to send email alert via Brevo API
void sendBrevoAlert(float temperature, float heartRate, float SpO2) {
HTTPClient http;

// Prepare JSON data for the email/SMS
String jsonData = "{";
jsonData += "\"sender\": {\"email\": \"vsbdhanu20@gmail.com\"},";
jsonData += "\"to\": [{\"email\": \"sriiiii2021@gmail.com\"}],";
jsonData += "\"htmlContent\": \"<html><body><h2>Warning!</h2><p>";
jsonData += "Temperature: " + String(temperature) + "°C<br>";
jsonData += "Heart Rate: " + String(heartRate) + " bpm<br>";
jsonData += "SpO2: " + String(SpO2);
jsonData += "}";

// Send HTTP request to Brevo API
http.begin(brevoAPI);
http.addHeader("Content-Type", "application/json");
http.addHeader("api-key", apiKey); // Set Brevo API key
int httpResponseCode = http.POST(jsonData);
```

```
if (httpResponseCode > 0) {
Serial.println("Alert sent via Brevo!");
} else {
Serial.println("Failed to send alert via Brevo");
}
http.end();
}
```

5. RESULTS AND DISCUSSION

5.1 OUTCOME:

The Cloud Integrated Patient Monitoring System successfully integrates various sensors with an ESP32 microcontroller to monitor and log patient health parameters in real time. The system reliably measures and transmits the following data to cloud platforms:

1. Body Temperature: The DHT11 sensor accurately monitors ambient temperature, providing readings in °C with a tolerance suitable for non-critical applications.
2. Heart Rate and SpO2 Levels: The MAX30100 pulse oximeter measures the patient's heart rate in beats per minute (bpm) and oxygen saturation (SpO2) percentage, ensuring real-time tracking of vital health metrics.
3. Cloud Integration: The system transmits sensor data to ThingSpeak for logging and visualization, enabling remote monitoring.
4. Alert Notifications: Abnormal readings trigger automated email alerts through the Brevo API, ensuring prompt action in case of emergencies.

5.2 CONNECTIONS:

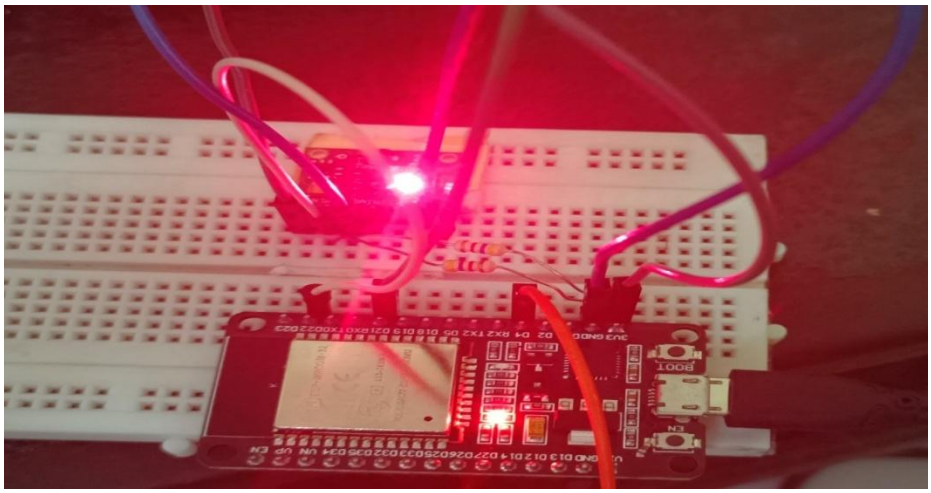

Figure-12 Practical Connection

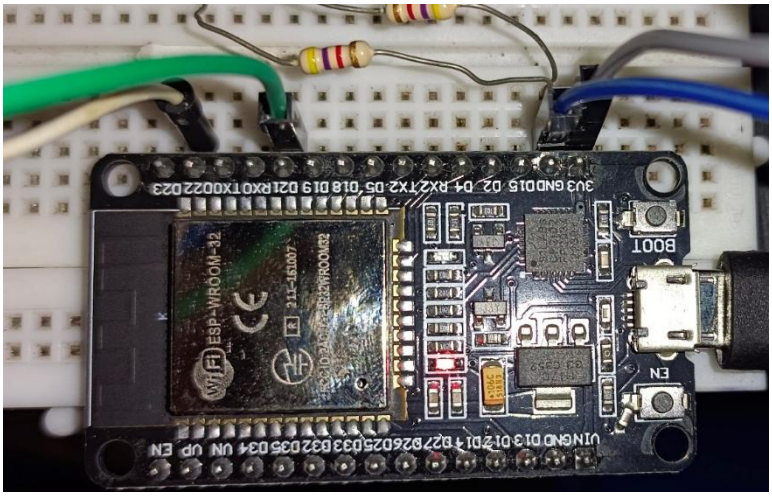

Figure-13 ESP32 Connections

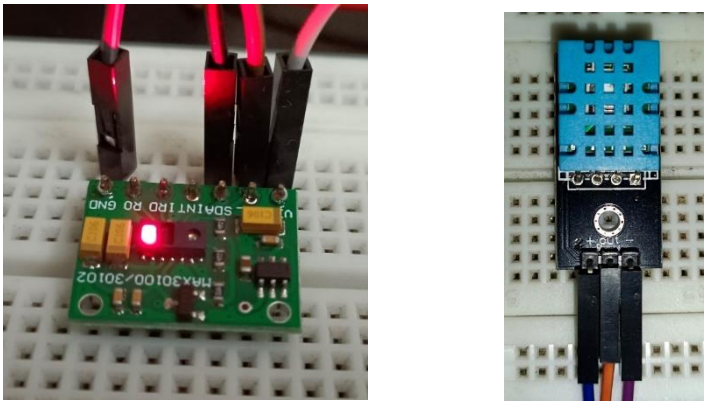

Figure-14 DHT11 and MAX30100/30102 used in the project

5.3 BODY TEMPERATURE, HEART RATE AND SpO2 READINGS:

Case-1:Normal reading(i.e. below threshold values)

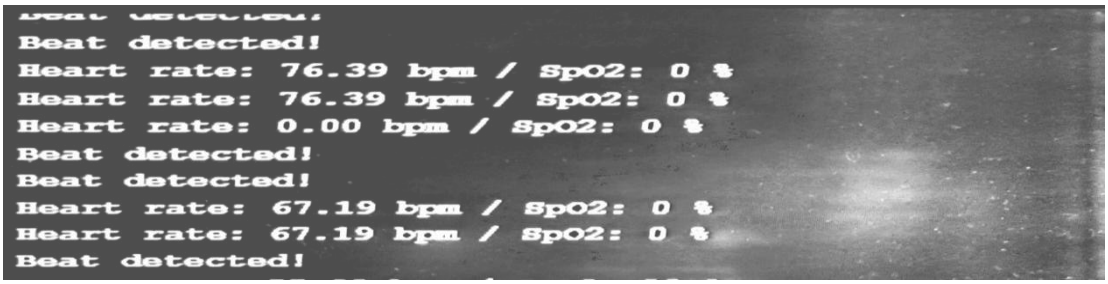

Figure-15 Heart rate and SpO2 readings

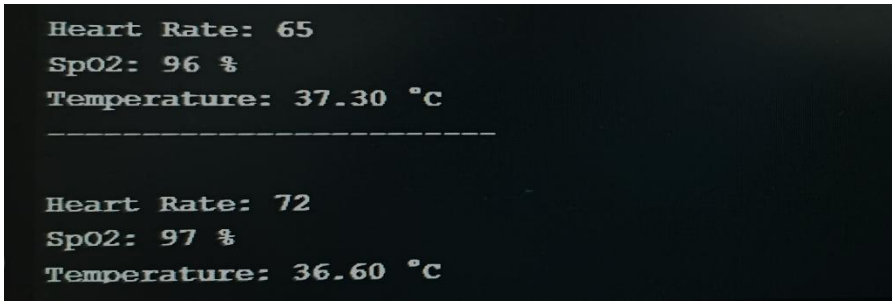

Figure-16 All three readings

Case-2:Preparing and sending email after the threshold value is exceeded

Figure-17 After exceeding threshold value

5.4 ALERT MESSAGE:

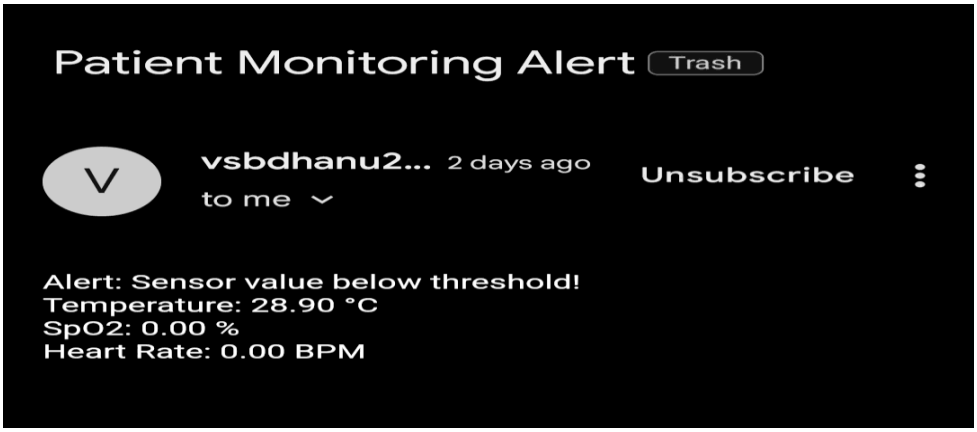

Figure-18 Message sent in Email

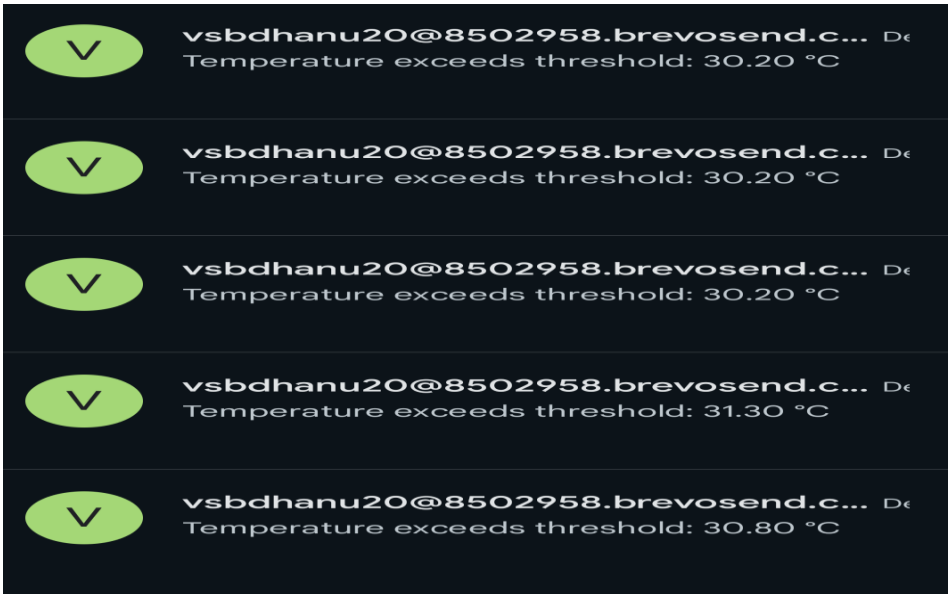

Figure-19 Continuous Message sent to Email for every 2 mins

5.5 CLOUD PLATFORM :

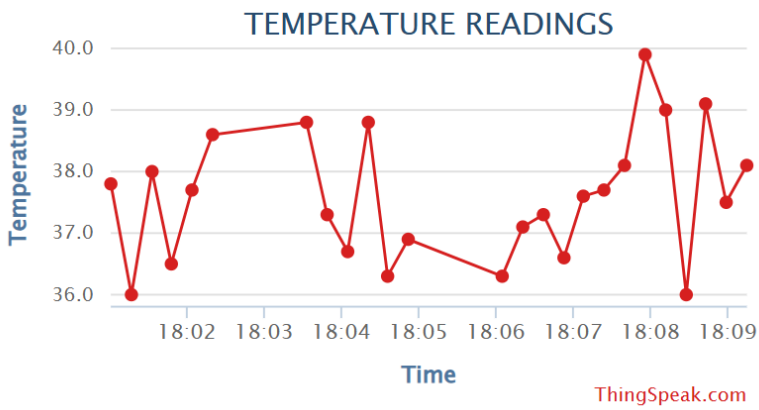

Figure-20 Temperature reading Graph from the cloud platform

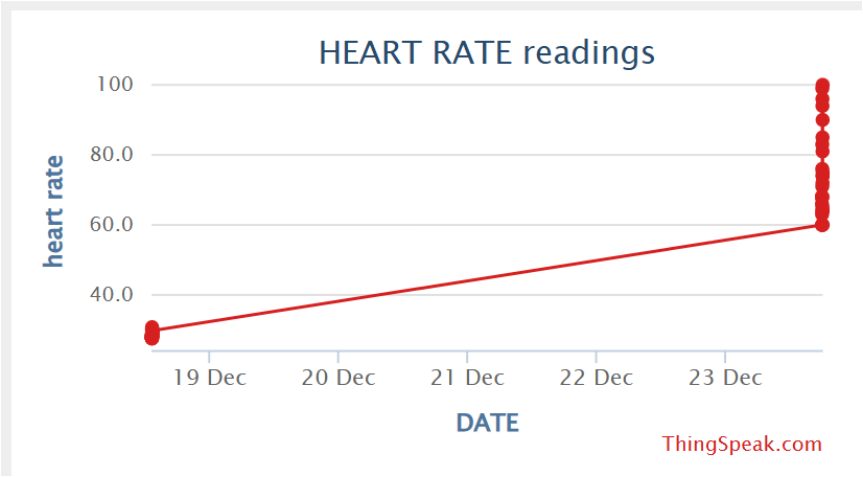

Figure-21 Heart rate Graph from the cloud platform

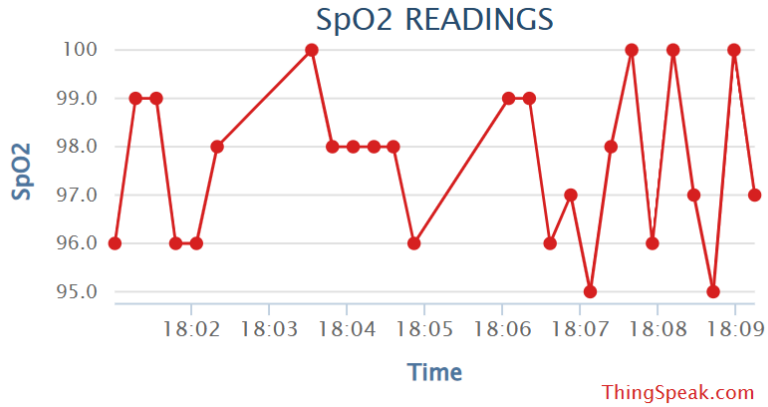

Figure-22 SpO2 readings Graph from the cloud platform

6. FUTURE SCOPE

The Cloud Integrated Patient Monitoring System has significant potential to evolve and make a substantial impact in the field of healthcare and IoT. As technology advances and healthcare systems increasingly adopt digital solutions, this project can be extended in various directions to enhance its utility and accessibility.

1. Integration with Advanced Sensors: Future iterations of the system can include advanced medical-grade sensors for more comprehensive monitoring, such as blood pressure, ECG, glucose levels, and respiratory rate. This will make the system a multi-parameter diagnostic tool capable of tracking a broader range of health metrics.
2. Mobile and Wearable Applications: By miniaturizing the hardware and integrating with wearable technology, the system can cater to patients who need continuous monitoring, such as those with chronic diseases or at risk of sudden health deterioration. Smartphone apps can further facilitate real-time monitoring and patient-doctor communication.
3. AI and Machine Learning Integration: Implementing artificial intelligence and machine learning algorithms could enable predictive analytics, identifying early signs of potential health issues. This would allow for timely interventions, reducing the risk of critical events and improving patient outcomes.
4. Enhanced Cloud Capabilities: The integration of more robust cloud platforms, such as AWS IoT or Google Cloud, can improve data security, scalability, and real-time data analysis. These platforms can also facilitate interoperability with hospital systems, ensuring seamless data sharing between patients, caregivers, and medical professionals.
5. Telemedicine and Remote Consultation: The system can be linked to telemedicine platforms, enabling healthcare providers to access patient data remotely and provide consultations based on real-time metrics. This is particularly beneficial for rural or underserved areas with limited access to healthcare facilities.

This project lays the groundwork for a scalable, cost-effective, and impactful healthcare solution. Its future iterations have the potential to revolutionize how patient data is monitored, shared, and acted upon, ultimately improving healthcare accessibility and quality worldwide.

7. REFERENCES

1. D. Akhil, M. V. Vardhan, K. V. Reddy, C. Preetham and N. Sangeeta, "Iot Based Icu Patient Monitoring Smart System," *2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, Vellore, India, 2024, pp. 1-6, doi: 10.1109/AIIoT58432.2024.10574787.

2. U. Gada, B. Joshi, S. Kadam, N. Jain, S. Kodeboyina and R. Menon, "IOT based Temperature Monitoring System," *2021 4th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)*, NaviMumbai, India, 2021, pp. 1-6, doi: 10.1109/ICNTE51185.2021.9487691.

3. V. S, M. Jagadeeswari, R. R, M. Balasenduran, A. U. S and T. Akileshwaran, "IoT Based Automatic Temperature Detection and Visitor Recognition System," *2023 4th International Conference on Smart Electronics and Communication (ICOSEC)*, Trichy, India, 2023, pp. 392-397, doi: 10.1109/ICOSEC58147.2023.10276022.

4. Jella Bharath Kumar, Banothu Charan , Guglavanth Vinod Kumar , Choudhary Sunil, Dr.P.Sumithabhashini," IoT Based Temperature Monitoring System UsingESP8266 & Blynk App", Volume 6, Issue 1, January-February 2024, E-ISSN: 2582-2160

5. Budijono, Santoso & Felita,"Smart Temperature Monitoring System Using ESP32 and DS18B20",*2021,IOP Conference Series: Earth and Environmental Science.*,794. 012125. 10.1088/1755-1315/794/1/012125.

6. Jeyavathana, Dr. Beulah and Ajay, Sakili and Srinivas, Doma Pavan, "Integration of Cloud-Based Patient Healthcare Monitoring System" (November 22, 2024)

7. Sultana, Salma & Rahman, Sadia & Rahman, Md & Chakraborty, Narayan & Hasan, Tanveer, 2021, "An IoT Based Integrated Health Monitoring System" ,549-554. 10.1109/ICCCA52192.2021.9666412.

8. C. Chakraborty and A. Kishor, "Real-Time Cloud-Based Patient-Centric Monitoring Using Computational Health Systems," in *IEEE Transactions on Computational Social Systems*, vol. 9, no. 6, pp. 1613-1623, Dec. 2022, doi: 10.1109/TCSS.2022.3170375.

9. D Prasanth, P Karthikeyan, N Yughesh, V Vishva, Department of Computer Science and Engineering, Madagadipet, Puducherry, India, "IOT-Based Remote Patient Monitoring System", International Journal Of Innovative Research In Technology, December 2024 | IJIRT | Volume 11 Issue 7 | ISSN: 2349-6002 IJIRT 170786

10. https://www.elprocus.com/a-brief-on-dht11-sensor/

11. https://r.search.yahoo.com/_ylt=Awr1WT24YmlnEAIAyIi7HAx.;_ylu=Y29sbwNzZzMEcG9zAzEEdnRpZAMEc2VjA3Ny/RV=2/RE=1736169400/RO=10/RU=https%3a%2f%2fwww.analog.com%2fmedia%2fen%2ftechnical-documentation%2fdata-sheets%2fMAX30100.pdf/RK=2/RS=piyVL2GNM5RugEkeSt4d6O2Q_74-

12. https://docs.arduino.cc/learn/starting-guide/the-arduino-software-ide/